

FastAPI 用户指导手册

1. 引言

通过“用户管理”样例，完成以下任务：

- 1. 3 个 REST API 请求；
- 2. 使用 `requirements.txt` 管理并安装 Python 软件包；
- 3. 使用 FastAPI 编写 1 个 GET 请求和 2 个 POST 请求；
- 4. 使用 SQLAlchemy 访问 MySQL 数据库中的 `users` 表；
- 5. 启动 FastAPI 后端服务，并通过 Postman 验证接口。

2. 三个 API 请求的设计结构

2.1 统一响应结构

接口成功时：

```
{
  "success": true,
  "message": "操作成功",
  "data": {}
}
```

接口失败时：

```
{
  "success": false,
  "message": "失败原因",
  "data": null
}
```

2.2 API 设计表

序号	请求方法	API 地址	功能	请求数据	是否访问 MySQL
1	GET	<code>/api/users/{user_id}</code>	根据用户 ID 查询用户	路径参数 <code>user_id</code>	是，读取 <code>users</code> 表
2	POST	<code>/api/users</code>	创建新用户	JSON 请求体	是，写入 <code>users</code> 表
3	POST	<code>/api/login</code>	验证用户名和密码	JSON 请求体	是，读取 <code>users</code> 表

2.3 HTTP 状态码设计

状态码	含义	使用场景
200	请求成功	查询用户成功、登录成功
201	创建成功	新用户创建成功
401	未通过身份验证	用户名或密码错误
404	资源不存在	指定用户不存在
409	数据冲突	用户名或邮箱已存在
422	参数校验失败	字段缺失、邮箱格式错误等
500	服务器内部错误	数据库操作异常

3. 项目目录结构

创建项目目录：

```
fastapi-user-api/  
├── main.py  
├── requirements.txt  
└── .env
```

进入项目目录：

```
mkdir fastapi-user-api  
cd fastapi-user-api
```

4. 配置 Python 虚拟环境

建议使用 Python 3.10 或更高版本。

4.1 创建虚拟环境

```
python -m venv .venv
```

4.2 激活虚拟环境

MacOS 或 Linux：

```
source .venv/bin/activate
```

Windows CMD：

```
.venv\Scripts\activate.bat
```

激活成功后，终端命令行前通常会出现 `(.venv)`。

5. requirements.txt

在项目根目录创建 `requirements.txt`：

```
fastapi
uvicorn[standard]
SQLAlchemy
PyMySQL
python-dotenv
email-validator
pwdlib[argon2]
```

其中：

软件包	作用
<code>fastapi</code>	创建 Web API
<code>uvicorn[standard]</code>	启动 ASGI 后端服务
<code>SQLAlchemy</code>	以 ORM 方式操作数据库
<code>PyMySQL</code>	连接 MySQL 数据库
<code>python-dotenv</code>	从 <code>.env</code> 文件读取环境变量
<code>email-validator</code>	支持 Pydantic 的邮箱格式校验
<code>pwdlib[argon2]</code>	对密码进行哈希和验证，避免保存明文密码

安装依赖：

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

6. 创建 MySQL 数据库

6.1 登录 MySQL

```
mysql -u root -p
```

6.2 创建数据库和专用账号

```
CREATE DATABASE IF NOT EXISTS fastapi_demo
  DEFAULT CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER IF NOT EXISTS 'fastapi_user'@'localhost'
  IDENTIFIED BY 'FastApi_123456';

GRANT ALL PRIVILEGES ON fastapi_demo.*
  TO 'fastapi_user'@'localhost';

FLUSH PRIVILEGES;
```

6.3 users 表结构

程序启动时会自动创建 `users` 表，对应表结构如下：

```
USE fastapi_demo;

CREATE TABLE IF NOT EXISTS users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

7. 配置数据库连接

在项目根目录创建 `.env` 文件：

```
DATABASE_URL=mysql+pymysql://fastapi_user:FastApi_123456@127.0.0.1:3306/fastapi_demo?
charset=utf8mb4
```

在 `.gitignore` 中加入：

```
.venv/
.env
__pycache__/
*.pyc
```

8. 编写 FastAPI 后端代码

在项目根目录创建 `main.py`：

```
from contextlib import asynccontextmanager
from datetime import datetime
import os
from typing import Generator
```

```

from dotenv import load_dotenv
from fastapi import Depends, FastAPI
from fastapi.encoders import jsonable_encoder
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from pydantic import BaseModel, EmailStr, Field
from pwdlib import PasswordHash
from sqlalchemy import DateTime, Integer, String, create_engine, func, or_, select
from sqlalchemy.exc import IntegrityError, SQLAlchemyError
from sqlalchemy.orm import (
    DeclarativeBase,
    Mapped,
    Session,
    mapped_column,
    sessionmaker,
)

# 读取 .env 文件
load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")
if not DATABASE_URL:
    raise RuntimeError("未配置 DATABASE_URL, 请检查 .env 文件")

# 创建数据库引擎和 Session maker
engine = create_engine(
    DATABASE_URL,
    pool_pre_ping=True,
    pool_recycle=3600,
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False,
)

class Base(DeclarativeBase):
    """SQLAlchemy ORM 模型基类。"""

class User(Base):
    """MySQL users 表对应的 ORM 模型。"""

    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    username: Mapped[str] = mapped_column(
        String(50), unique=True, nullable=False, index=True
    )

```

```

email: Mapped[str] = mapped_column(
    String(100), unique=True, nullable=False, index=True
)
password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
created_at: Mapped[datetime] = mapped_column(
    DateTime, nullable=False, server_default=func.now()
)

```

```

class UserCreate(BaseModel):

```

```

    """创建用户时的 JSON 请求体 """

```

```

    username: str = Field(
        min_length=3,
        max_length=50,
        pattern=r"^[A-Za-z0-9_]+$",
        description="只能包含英文字母、数字和下划线",
    )
    email: EmailStr
    password: str = Field(min_length=6, max_length=128)

```

```

class LoginRequest(BaseModel):

```

```

    """登录时的 JSON 请求体 """

```

```

    username: str = Field(min_length=3, max_length=50)
    password: str = Field(min_length=6, max_length=128)

```

```

password_helper = PasswordHash.recommended()

```

```

def get_db() -> Generator[Session, None, None]:

```

```

    """为每次请求创建独立的数据库 Session, 并在请求结束后关闭 """

```

```

    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

```

def success_response(message: str, data=None, status_code: int = 200) -> JSONResponse:

```

```

    """生成统一的成功 JSON 响应 """

```

```

    return JSONResponse(
        status_code=status_code,
        content=jsonable_encoder(
            {
                "success": True,
                "message": message,
                "data": data,
            }
        )
    )

```

```

        }
    ),
)

def error_response(message: str, status_code: int, data=None) -> JSONResponse:
    """生成统一的失败 JSON 响应。"""

    return JSONResponse(
        status_code=status_code,
        content=jsonable_encoder(
            {
                "success": False,
                "message": message,
                "data": data,
            }
        ),
    )

@asynccontextmanager
async def lifespan(app: FastAPI):
    """服务启动时创建尚不存在的数据表。"""

    Base.metadata.create_all(bind=engine)
    yield

app = FastAPI(
    title="FastAPI 用户管理示例",
    version="1.0.0",
    description="包含一个 GET 请求、两个 POST 请求以及 MySQL 数据访问。",
    lifespan=lifespan,
)

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc: RequestValidationError):
    """将 FastAPI 参数校验错误转换成统一的失败 JSON。"""

    return error_response(
        message="请求参数校验失败",
        status_code=422,
        data=jsonable_encoder(exc.errors()),
    )

# API 1: GET, 根据用户 ID 查询 MySQL users 表
@app.get("/api/users/{user_id}", tags=["用户"])
def get_user(user_id: int, db: Session = Depends(get_db)):
    try:
        user = db.scalar(select(User).where(User.id == user_id))

```

```

except SQLAlchemyError:
    return error_response("数据库查询失败", 500)

if user is None:
    return error_response("用户不存在", 404)

return success_response(
    "查询用户成功",
    {
        "id": user.id,
        "username": user.username,
        "email": user.email,
        "created_at": user.created_at,
    },
)

```

API 2: POST, users 表新增用户

```
@app.post("/api/users", tags=["用户"])
```

```
def create_user(payload: UserCreate, db: Session = Depends(get_db)):
```

```
    try:
```

```
        existing_user = db.scalar(
            select(User).where(
                or_(
                    User.username == payload.username,
                    User.email == str(payload.email),
                )
            )
        )
```

```
    if existing_user is not None:
        return error_response("用户名或邮箱已存在", 409)
```

```
    user = User(
        username=payload.username,
        email=str(payload.email),
        password_hash=password_helper.hash(payload.password),
    )
```

```
    db.add(user)
    db.commit()
    db.refresh(user)
```

```
    return success_response(
        "用户创建成功",
        {
            "id": user.id,
            "username": user.username,
            "email": user.email,
            "created_at": user.created_at,
        },
        status_code=201,
    )

```



```

    )
except IntegrityError:
    db.rollback()
    return error_response("用户名或邮箱已存在", 409)
except SQLAlchemyError:
    db.rollback()
    return error_response("数据库写入失败", 500)

# API 3: POST, 从 users 表读取用户并验证密码
@app.post("/api/login", tags=["认证"])
def login(payload: LoginRequest, db: Session = Depends(get_db)):
    try:
        user = db.scalar(
            select(User).where(User.username == payload.username)
        )
    except SQLAlchemyError:
        return error_response("数据库查询失败", 500)

    if user is None or not password_helper.verify(
        payload.password, user.password_hash
    ):
        return error_response("用户名或密码错误", 401)

    return success_response(
        "登录验证成功",
        {
            "id": user.id,
            "username": user.username,
            "email": user.email,
        },
    )

```

9. 启动 FastAPI 后端服务

启动服务：

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

- `main`：表示 `main.py` 文件；
- `app`：表示代码中的 `app = FastAPI(...)` 对象；
- `--host 0.0.0.0`：允许从本机以外的设备访问；
- `--port 8000`：服务端口为 8000；
- `--reload`：代码变化后自动重启，仅建议用于开发环境。

终端出现类似信息表示启动成功：

```
Uvicorn running on http://0.0.0.0:8000
Application startup complete.
```

本机访问地址：

```
http://127.0.0.1:8000
```

FastAPI 自动生成的 Swagger 接口文档：

```
http://127.0.0.1:8000/docs
```

ReDoc 接口文档：

```
http://127.0.0.1:8000/redoc
```

10. Postman 验证API

在 Postman 中创建一个 Collection，例如：

```
FastAPI User API
```

设置环境变量：

变量名	初始值
base_url	http://127.0.0.1:8000

请求地址可以写成：

```
{{base_url}}/api/users
```

10.1 验证 POST `/api/users`

请求设置：

```
Method: POST  
URL: {{base_url}}/api/users
```

选择：

```
Body → raw → JSON
```

输入：

```
{
  "username": "student01",
  "email": "student01@example.com",
  "password": "12345678"
}
```

成功结果: HTTP 201 Created

```
{
  "success": true,
  "message": "用户创建成功",
  "data": {
    "id": 1,
    "username": "student01",
    "email": "student01@example.com",
    "created_at": "2026-06-25T10:00:00"
  }
}
```

再次提交相同数据, 失败结果: HTTP 409 Conflict

```
{
  "success": false,
  "message": "用户名或邮箱已存在",
  "data": null
}
```

10.2 验证 GET /api/users/{user_id}

使用创建用户接口实际返回的 id

```
Method: GET
URL: {{base_url}}/api/users/1
```

成功结果: HTTP 200 OK

```
{
  "success": true,
  "message": "查询用户成功",
  "data": {
    "id": 1,
    "username": "student01",
    "email": "student01@example.com",
    "created_at": "2026-06-25T10:00:00"
  }
}
```

查询不存在的用户:

```
Method: GET
URL: {{base_url}}/api/users/999999
```

失败结果: HTTP 404 Not Found

```
{
  "success": false,
  "message": "用户不存在",
  "data": null
}
```

10.3 验证 POST /api/login

请求设置:

```
Method: POST
URL: {{base_url}}/api/login
```

选择:

Body → raw → JSON

输入正确账号和密码:

```
{
  "username": "student01",
  "password": "12345678"
}
```

成功结果: HTTP 200 OK

```
{
  "success": true,
  "message": "登录验证成功",
  "data": {
    "id": 1,
    "username": "student01",
    "email": "student01@example.com"
  }
}
```

输入错误密码:

```
{
  "username": "student01",
  "password": "wrong_password"
}
```

失败结果: HTTP 401 Unauthorized

```
{
  "success": false,
  "message": "用户名或密码错误",
  "data": null
}
```